Phân công công việc nhóm

Mục tiêu

Hoàn thiện project **Multi-AZ Fault-Tolerant Event-Driven Autoscaling for Microservices on Amazon EKS** dựa trên code Terraform hạ tầng hiện có. Phạm vi tập trung vào SQS, KEDA, khả năng chịu lỗi, observability và chaos engineering.

> *Terraform đã có code là nền tảng để tiếp tục tích hợp; điều đó không đồng nghĩa tài nguyên AWS đã được triển khai hoặc xác minh. Trước khi chạy* `terraform apply` *, nhóm cần review plan, quyền truy cập và chi phí.*

Phân công

Thành viên	Vai trò	Phạm vi chính	Bàn giao
Khánh — Trưởng nhóm	Tích hợp Kubernetes, KEDA và điều phối	Review code/outputs Terraform hiện có; hoàn thiện manifests trong `k8s/` ; kết nối ServiceAccount với IRSA, queue và image; triển khai, kiểm thử tích hợp; review PR và theo dõi tiến độ.	Các manifests triển khai được; API đi qua ALB; Worker nhận việc từ SQS; KEDA scale Worker theo backlog; cấu hình topology và hướng dẫn chạy hệ thống.
Phát: _____	Backend	Hoàn thiện `apps/api-service/` và `apps/worker-service/` ; API gửi job vào SQS; Worker poll và xử lý message; cấu hình	API và Worker build được thành image; luồng gửi–nhận–xử lý job hoạt động; hướng dẫn cấu hình môi trường và kết quả kiểm thử.

		retry, xử lý message trùng theo hướng idempotent, graceful shutdown; hoàn thiện Dockerfile và kiểm thử luồng ứng dụng.	
Khoa: _____	Observability, đo lường và thí nghiệm	Hoàn thiện cấu hình Prometheus, Grafana, Loki; chuẩn bị dashboard; xây workload k6; thu thập các mốc T0–T6 và metrics; viết kịch bản, lưu evidence và phối hợp chạy chaos test an toàn.	Workload có thể chạy lại; dashboard và dữ liệu đo; kết quả baseline/scaling/recovery; evidence và cập nhật tài liệu thí nghiệm.

Phạm vi công việc chi tiết

1. Khánh — trưởng nhóm, Kubernetes và tích hợp

- Kiểm tra Terraform hiện có trong `infra/` : module VPC/EKS, SQS, ECR, IRSA, outputs và biến môi trường cần thiết cho ứng dụng.
- Xác minh trạng thái môi trường trước khi triển khai: Terraform plan, quyền EKS, kubeconfig, kết nối SQS và khả năng cấp quyền IRSA. Không đưa credentials, state hoặc tfvars có thông tin nhạy cảm vào Git.
- Hoàn thiện các manifest trong `k8s/base/` , `k8s/config/` , `k8s/autoscaling/` và `k8s/topology/` : namespace, API/Worker Deployment, Service, Ingress, ServiceAccount, KEDA ScaledObject, HPA baseline và topology spread.
- Áp dụng contract tích hợp ban đầu ở mục dưới vào ConfigMap, Deployment, Service, KEDA và tài liệu triển khai.
- Ghép các phần theo thứ tự: deploy ứng dụng xác minh API/SQS/Worker xác minh KEDA scale kiểm tra phân bố Pod giữa các AZ.
- Review thay đổi, ghi lại lệnh triển khai/kiểm tra và tổng hợp kết quả nhóm.

2. Thành viên 2 — Backend

- API nhận request tạo job, validate input cần thiết và gửi message vào SQS.
- Worker nhận message, xử lý job, chỉ xóa message sau khi xử lý thành công và ghi log có message/job ID cùng timestamp.
- Xử lý redelivery và message trùng an toàn; cấu hình thời gian xử lý và shutdown để Worker có thể hoàn tất hoặc dừng công việc đang xử lý một cách có kiểm soát khi nhận SIGTERM.
- Hoàn thiện dependency, Dockerfile và hướng dẫn chạy local/container.
- Bàn giao danh sách biến môi trường, quyền SQS Worker/API cần dùng và cách kiểm thử end-to-end.

Contract tích hợp ban đầu

Các thành viên dùng chung cấu hình sau để phát triển và tích hợp. Nếu code Terraform hoặc cấu hình triển khai đang khác, Khánh điều phối cập nhật để chỉ còn một giá trị thống nhất; không tự ý tạo queue mới.

Hạng mục	**Quy ước ban đầu**
API tạo job	`POST /jobs`
Health check API	`GET /health`
Port API	`8080` (container và Service target port)
Queue logic	`jobs-queue`; URL thực tế lấy từ Terraform output và truyền qua `QUEUE_URL`
Region	`ap-southeast-1` qua `AWS_REGION`, đồng bộ với cấu hình `infra/`
Biến môi trường API	`PORT=8080`, `AWS_REGION`, `QUEUE_URL`
Biến môi trường Worker	`AWS_REGION`, `QUEUE_URL`
Định dạng message	JSON gồm `job_id` và `payload`
Quyền SQS của API	`sqs:SendMessage` trên ARN của main queue

Quyền SQS của Worker	`sqs:ReceiveMessage`, `sqs:DeleteMessage` và `sqs:GetQueueAttributes` trên ARN của main queue
Image tag	`<git-sha>` cho cả API và Worker; dùng ECR URL Terraform output tương ứng (`ecr_api_url`, `ecr_worker_url`)

Ví dụ message gửi vào SQS:

```
{
  "job_id": "8f6b4e70-0af3-4f9a-ae72-4e319c73187a",
  "payload": {
    "example": "job data"
  }
}
```

`job_id` phải duy nhất cho mỗi job; Worker ghi nhận ID này trong log để hỗ trợ đối chiếu retry, redelivery và xử lý trùng. `payload` chứa dữ liệu nghiệp vụ của job.

Quy ước image tag bất biến giúp xác định chính xác phiên bản đang chạy. Không dùng `latest` làm tag triển khai chung. Tag image theo Git commit SHA và cấu hình cùng tag đó trong Kubernetes manifests.

> ***Cần đồng bộ cấu hình hiện có:*** *`k8s/config/configmap.yaml` hiện khai báo `QUEUE_NAME: "worker-queue"` và `AWS_REGION: "us-west-2"`, chưa khớp contract ban đầu và region của hạ tầng. Khi hoàn thiện manifest, dùng `jobs-queue` làm tên logic, `ap-southeast-1` làm region và `QUEUE_URL` lấy từ Terraform output. Terraform SQS tạo tên queue từ biến `queue_name`, vì vậy xác nhận biến đó là `jobs-queue` trước khi triển khai.*

3. Thành viên 3 — Observability, đo lường và thí nghiệm

- Hoàn thiện cấu hình thu thập metrics và hướng dẫn cài Prometheus/Grafana/Loki.
- Tạo dashboard theo dõi queue backlog, số Worker replicas, throughput, processing latency và trạng thái Pod.
- Hoàn thiện k6 workload với tham số tải có thể cấu hình; lưu lại cấu hình và cách chạy để tái lập kết quả.
- Định nghĩa và thu thập mốc thời gian:
  - T0: workload/job được tạo.
  - T1: message xuất hiện trong SQS.
  - T2: KEDA quan sát scaling signal.

    - T3: desired replica count thay đổi.
    - T4: Worker Pod được tạo.
    - T5: Worker Pod Ready.
    - T6: message được xử lý thành công.
- Tính latency từ dữ liệu quan sát được; nêu rõ nguồn dữ liệu và giới hạn sai số, không kết luận trước bottleneck.
- Chạy thử nghiệm theo thứ tự an toàn: baseline tải tăng và scale Pod failure. Chỉ chạy Node/AZ failure sau khi nhóm thống nhất cách ly môi trường, điều kiện dừng và kiểm soát chi phí.
- Lưu log, CSV, ảnh dashboard và tóm tắt vào `docs/evidence/` ; cập nhật `docs/experiments.md` , `docs/fault-tolerance.md` và tài liệu liên quan.

Mốc phối hợp và tiêu chí hoàn thành

Mốc	Người phối hợp	Điều kiện hoàn thành
1. Chốt contract	Khánh + Backend	Thống nhất queue, biến môi trường, API contract, message schema, port và image tag.
2. Hoàn thiện ứng dụng	Backend	API đưa được job vào SQS; Worker nhận và xử lý job; có log và hướng dẫn build/run.
3. Deploy nền tảng ứng dụng	Khánh + Backend	Manifests chạy được; ServiceAccount/IRSA đúng; API, SQS và Worker kết nối được.
4. Xác minh autoscaling	Khánh + thành viên 3	KEDA đọc đúng queue; tải tăng làm Worker scale; ghi lại backlog và replica count.
5. Đo lường và fault tolerance	Thành viên 3 + cả nhóm	Có baseline, kết quả thử Pod failure và evidence; lỗi được kiểm soát, message không bị mất ngoài dự kiến.
6. Tích hợp cuối	Cả nhóm, Khánh điều phối	Tài liệu cập nhật; hướng dẫn chạy/kiểm thử rõ ràng; PR đã được review; không có secret hoặc state nhạy cảm trong thay đổi.

Nguyên tắc làm việc

- Không triển khai lại hoặc sửa lớn Terraform nếu chưa review code và thống nhất với Khánh; báo rõ khi thay đổi có thể tạo tài nguyên hoặc phát sinh chi phí.

- Mọi thử nghiệm cần ghi mục tiêu, tải, điều kiện failure, metrics cần thu, cách dừng và cách khôi phục.
- Chạy kiểm tra từng lớp trước khi ghép toàn hệ thống; ghi lại lệnh và kết quả.
- Tạo nhánh riêng cho từng phần, dùng Pull Request để review; không commit credentials, Secret thật, `.tfstate` hoặc `.tfvars` chứa secret.
- Không thêm công nghệ lớn khi chưa thống nhất; **không dùng Karpenter và không đưa node autoscaling vào phạm vi nghiên cứu**. Managed Node Groups chỉ cung cấp capacity nền cho EKS.
- Khánh theo dõi phụ thuộc và tiến độ, hỗ trợ gỡ blocker, review thay đổi và xác nhận tiêu chí bàn giao của từng mốc.